

Supporting Information: An Open-Source Cost-Effective Arduino Platform for High-Throughput Solar Device Stability Testing

Juan-Pablo Correa-Baena

CONTENTS

I	Hardware Implementation Specifications	2
I-A	PCB schematics and layout diagrams	2
I-B	Hardware Configuration Constants	2
I-C	Arduino microcontroller specifications	3
I-D	DAC and ADC component datasheets and configurations	3
I-E	3D-Printed Components and Sample Holder	3
II	Software Details	3
II-A	Software Architecture and Framework	3
II-B	Hardware Communication Protocol	3
II-C	Data Processing and Storage	4

I. HARDWARE IMPLEMENTATION SPECIFICATIONS

A. PCB schematics and layout diagrams

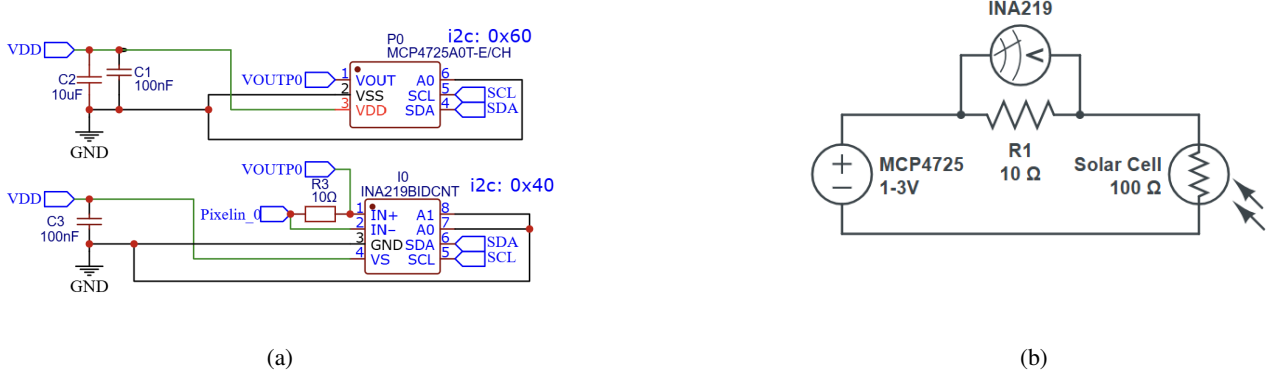


Fig. 1: Electrical diagrams and component layouts of the stability measurement system. (a) Detailed schematic of a single measurement channel, showing the DAC, ADC, and shunt resistor configuration. (b) Simplified circuit diagram of single device measurement.

The main board of the Stability Measurement System (SMS) was designed in EasyEDA, with fabrication and assembly handled by JLCPCB [?]. All design files, including schematics and PCB layouts, are open-source to facilitate replication and modification by other researchers. The board is engineered to test up to eight solar devices in parallel, with each pixel monitored by an independent sub-circuit.

The core of each sub-circuit, detailed in the schematics in Figures 1a and 1, consists of two primary integrated circuits (ICs) that operate over an I²C bus:

- **MCP4725 DAC:** A digital-to-analog converter used to apply a precise voltage bias to the device. Each of the eight DACs is assigned a unique I²C address from 0x60 to 0x67. The DAC can supply a voltage from 0 to 3.3V with a 0.8 mV resolution. Note that each specific version of the MCP4725 chip used can only support two I2c addresses, so four unique MCP4725 chips were used to achieve the full I2c range required.
- **INA219 ADC:** A high-side current and voltage sensor that monitors the device's output. Each of the eight ADCs is assigned a unique I²C address from 0x40 to 0x47. The sensor measures the voltage drop across a 10 Ω shunt resistor to determine the current. This configuration provides a current measurement range of 0 to 32 mA with a resolution of 8 μA and a voltage measurement range of 0 to 27 V with a resolution of 4 mV.

In the measurement circuit (Figure 1a), the MCP4725 DAC (P0) applies a variable load voltage, ranging from 0 to 3.3 V with a 12-bit resolution (approximately 0.8 mV per step), to the connected solar device. The INA219 ADC (I0) then measures the resulting voltage and current. Current is measured by monitoring the voltage drop across a 10Ω shunt resistor (R3). This configuration allows for current measurements in the range of 0-32 mA with a resolution of 8 μA. To ensure stable power delivery and reduce signal noise, decoupling capacitors (C1, C2, C3) are placed on the voltage input line.

The entire measurement process for all eight channels is orchestrated by an Arduino Nano [?] (AN), which communicates with the DAC and ADC components via the I²C protocol. The selection of the MCP4725 and INA219 components was based on their low cost, high availability, and robust software support, with well-documented C++ libraries provided by Adafruit [?]. For applications requiring different voltage and current ranges, the shunt resistor can be swapped easily.

B. Hardware Configuration Constants

The firmware relies on hard-coded constants derived from the physical PCB design. These values are defined in the `sensor.h` header file and must be updated if the hardware bill of materials is modified.

- **Shunt Resistor (R):** 10 Ω. This value dictates the current sensing range of the INA219 ADC. With a maximum shunt voltage of 320 mV, the theoretical current limit is 32 mA.
- **Logic Level (V_{DD}):** 3.3 V. This reference voltage is used for DAC calculations to convert the 12-bit digital value (0–4095) into an analog output voltage.
- **I2C Addressing:** The system assumes a contiguous block of I2C addresses for the ADCs (0x40 – 0x47) and DACs (0x60 – 0x67), corresponding to channels 1 through 8 respectively.

C. Arduino microcontroller specifications

The system is controlled by an Arduino Nano [?], selected for its compact form factor, widespread availability, and extensive community support. The Nano is based on the ATmega328P microcontroller [?], which provides sufficient computational power for managing the measurement tasks.

While the system was designed around the Arduino Nano, it is compatible with other Arduino boards. The primary requirements for a substitute board are sufficient flash memory to accommodate the firmware and support for the I²C communication protocol [?], which requires at least one set of SDA and SCL pins. Boards such as the Arduino Uno or Arduino Mega meet these criteria and could be used with minimal modification to the firmware's pin definitions.

D. DAC and ADC component datasheets and configurations

TABLE I: Bill of materials and cost breakdown for five Stability Measurement System (SMS) modules. PCB prices include fabrication, component sourcing, and assembly. Users may source parts and perform assembly manually; however, this requires proficiency in surface-mount device (SMD) soldering.

Component	Qty	Unit Cost (\$)	Total Cost (\$)
Arduino Nano (or compatible clone)	5	7.00	35.00
Main PCB assembly	5	40.50	202.50
Holder PCB	5	11.40	57.00
3D-printed enclosure (ABS)	5	4.00	20.00
LED grow light	1	60.00	60.00
Cables and connectors	10	2.00	20.00
Grand Total			394.50

E. 3D-Printed Components and Sample Holder

It is recommended that at least the top surface of the sample holder be printed out of ASA, PC, or some similarly heat resistant material, as printing out of regular PLA will result in warping after several hours of use under continuous illumination.

Our batch of sample holders from JLCPCB showed some variation in quality of pogo pin assembly, with about a third of the PCBs having unstable pogo pins, which resulted in failed connections to the solar devices. It may be worth it for the user to order the sample holder PCB with the pogo pins un-assembled and complete the soldering themselves.

II. SOFTWARE DETAILS

The software ecosystem follows a host-controller architecture, consisting of a high-level Python application for the host computer and embedded C++ firmware for the Arduino microcontrollers. The Python implementation leverages PySerial [?] for asynchronous hardware communication, NumPy [?] for efficient array manipulation, and Matplotlib [?] for real-time data visualization.

A. Software Architecture and Framework

The host control software is built upon the PySide6 [?] framework, which provides a responsive graphical user interface (GUI). The application employs a multi-threaded design pattern to ensure scalability. The main execution thread manages user interactions and interface updates, while the business logic spawns independent worker threads for each connected microcontroller. This separation ensures that the data acquisition and logging processes for one device do not block the execution of others, enabling the simultaneous operation of numerous devices on a single host machine.

B. Hardware Communication Protocol

Data exchange between the host and the microcontrollers relies on USB virtual COM ports using a lightweight, text-based serial protocol. The host initiates experiments by transmitting a single command string containing the operational parameters (e.g., voltage limits, step sizes, and integration times) formatted as comma-separated values. Upon reception, the firmware parses these parameters and enters an autonomous execution loop.

By offloading precise timing and feedback control to the embedded hardware, the system eliminates latency artifacts associated with host operating system scheduling or USB communication overhead. Measurement results are streamed asynchronously back to the host in real-time as they are acquired. This architecture ensures that the microcontroller's processing power is dedicated strictly to maintaining stable sampling intervals and executing the control algorithm, rather than handling high-frequency interrupt requests for individual voltage steps.

C. Data Processing and Storage

The software implements a buffered I/O strategy to handle long-duration datasets efficiently. As raw data lines are received from the microcontroller, they are parsed and appended to a local NumPy array buffer. Once the buffer size exceeds a predefined threshold (currently set to 20 lines), the data is flushed and appended to a CSV file on the disk. This chunked writing mechanism prevents memory exhaustion during stability tests that can last hundreds of hours.

To optimize real-time visualization performance, the system generates a secondary "compressed" dataset alongside the raw file during MPPT trials. This compressed file stores a single averaged data point per device representing the statistical mean of each flushed buffer. By reducing the data density significantly, the plotting engine can render long-term performance traces without the latency or stability issues associated with processing millions of raw data points, while the full-resolution data remains available for post-hoc analysis.